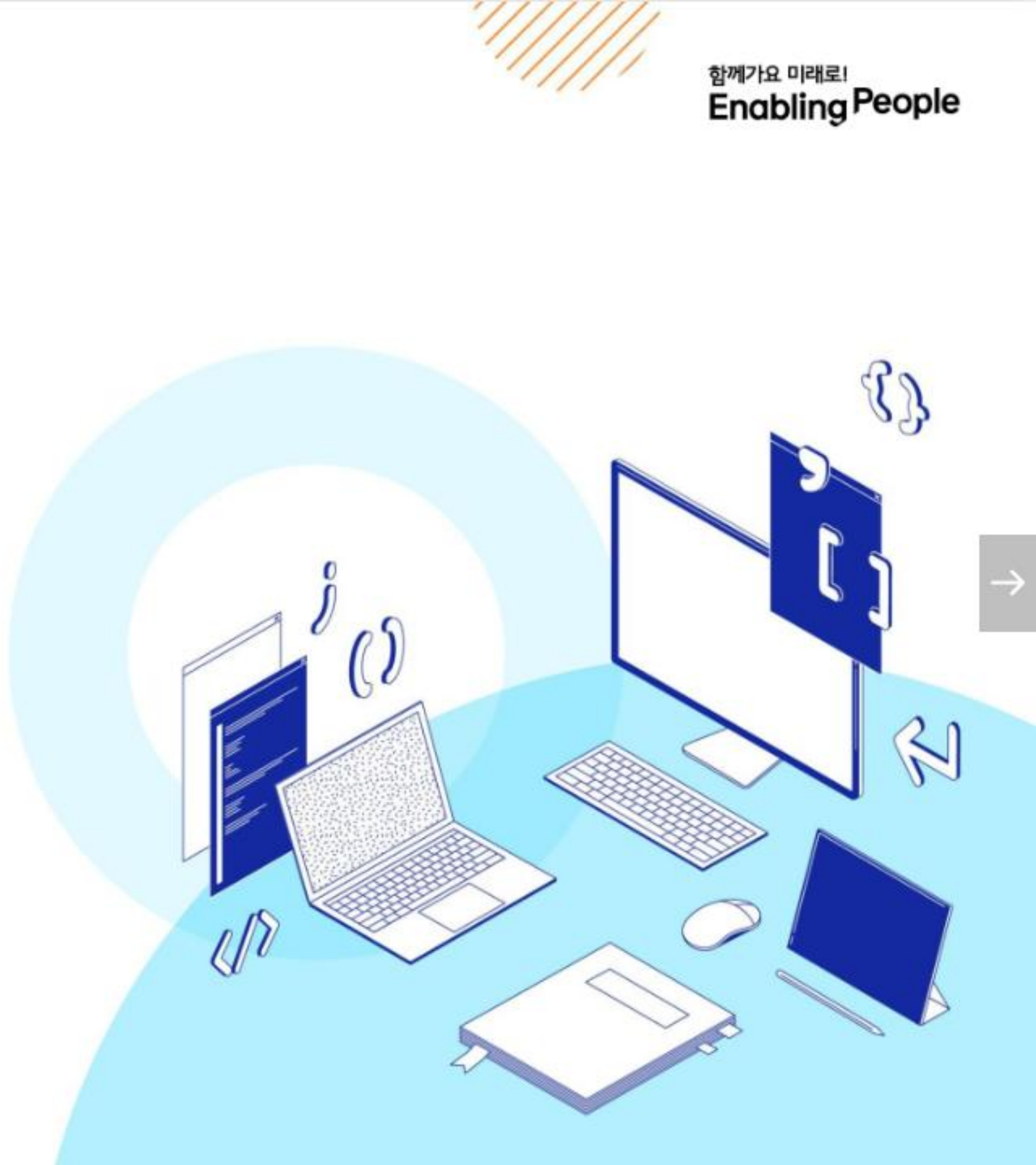


SSAFY

Database

Many To One Relationships 02

Python



목 차

Article & User

- 모델 관계 설정
- 게시글 CREATE
- 게시글 READ
- 게시글 UPDATE
- 게시글 DELETE

Comment & User

- 모델 관계 설정
- 댓글 CREATE
- 댓글 READ
- 댓글 DELETE

목 차

View decorators

- Allowed HTTP methods

ERD

- ERD 구성 요소
- ERD 제작 사이트

참고

- 추가 기능 구현

학습 시작

사용자 인증 기반 게시글과 댓글 기능을 구현해봅시다.

- 사용자와 게시글, 댓글은 어떤 관계를 가지고 참조 필드를 어느 모델 클래스에 작성하는지 고민해봅시다.

1. 각각  관계를 가지도록 모델을 설계합니다.
 - 유저 (): 게시글 () / 유저 (): 댓글 ()
2. User 모델은  을 참조해 게시글과 댓글 작성자를  를 사용하여 연결합니다.
3. Django의  을 활용해 Article과 Comment 모델의 CRUD기능을 구현합니다.

- View decorators를 사용해 허용된 HTTP 요청만 처리하는 방식도 학습합니다.
- ERD를 통해 모델 간 관계를 시각적으로 정리하고, 구조를 한눈에 파악할 수 있습니다.
- 인증된 사용자만 댓글 작성 및 삭제가 가능하도록 기능을 제한하는 방식도 실습합니다.

학습 목표

- ✓ User와 Article, User와 Comment 모델 간의 N:1 관계를 정의할 수 있다.
- ✓ 인증된 사용자만 게시글/댓글을 작성하거나 삭제할 수 있도록 구현할 수 있다.
- ✓ Django의 ORM을 활용해 CREATE/READ/UPDATE/DELETE 기능을 실습할 수 있다.
- ✓ View decorators를 통해 HTTP 요청 방식에 따라 기능을 제어할 수 있다.
- ✓ ERD를 활용해 모델 관계 구조를 시각적으로 표현하고 설명할 수 있다.

Article & User

모델 관계 설정

Article - User 모델 관계 설정

- User 외래 키 정의

```
# articles/models.py
from django.conf import settings  ※ User 모델을 직접 import 해서 사용하지 않음을 유의

class Article(models.Model):
    user = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.CASCADE)
    title = models.CharField(max_length=10)
    content = models.TextField()
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)
```

TIP

User 모델을 직접 Import 하지 않는 이유

- Article 클래스 생성 시점이 User 클래스 시점보다 빠른 경우 외래 키 설정에서 참고할 User 모델을 찾지 못해 에러가 발생할 수 있어요.
- models.py 에서는 안정적인 참조가 필요하기 때문에 settings.AUTH_USER_MODEL 의 값을 사용해요.

User 모델을 참조하는 2가지 방법

- `settings.AUTH_USER_MODEL`
 - `settings.py` 에서 정의된 `AUTH_USER_MODEL` 설정 값을 가져옴
 - 반환 값 : 'accounts.User' (문자열)
 - `models.py` 에서 User 모델을 참조할 때 주로 사용
- `get_user_model()`
 - 현재 `settings.py` 에 정의되어 활성화된 User 모델을 가져옴
 - 반환 값 : User Object (객체)
 - `models.py` 를 제외한 다른 모든 위치에서 사용

TIP

`settings.AUTH_USER_MODEL`의 반환 값이 문자열이어도 괜찮은 이유

- 모델의 경로 형태인 문자열이 `ForeignKey`의 참조 모델로 설정되면, Django 에서 내부적으로 해당 모델이 완전히 로딩된 후 모델 클래스를 가져와 처리하는 지연 평가(lazy evaluation) 방식으로 동작하기 때문입니다.

Migration (1/3)

- 기존에 테이블이 있는 상황에서 필드를 추가하려고 하기 때문에 발생하는 과정
- 기본적으로 모든 필드에는 NOT NULL 제약 조건이 설정되어 있어, 데이터 없이 새로운 필드를 추가할 수 없음
- '1'을 입력하고 Enter 진행 (다음 화면에서 직접 기본 값 입력)

```
$ python manage.py makemigrations
```

It **is** impossible to add a non-nullable field '**user**' to article without specifying a default.
This **is** because the database needs something to populate existing rows.

Please select a **fix**:

1) Provide a one-off default now (will be **set** on **all** existing rows **with** a null value **for** this column)

2) Quit **and** manually define a default value **in** models.py.

Select an **option**:

Migration (2/3)

- 추가하는 외래 키 필드에 어떤 데이터를 넣을 것인지 직접 입력해야 함
- 마찬가지로 '1'을 입력하고 Enter 진행
- 기존에 작성된 게시글이 있다면 모두 1번 회원이 작성한 것으로 처리됨

Please enter the default value **as** valid Python.
The datetime **and** django.utils.timezone modules are available, so it **is** possible to provide
e.g. timezone.now **as** a value.
Type '**exit**' to **exit** this prompt

- ※ 1번 회원이 없는 경우 migrate 시 에러 발생할 수 있음을 유의

Migration (3/3)

- migrations 파일 생성 후 migrate 진행

```
$ python manage.py migrate
```

- articles_article 테이블에 user_id 필드 생성 확인

	id	title	content	created_at	updated_at	user
	INTEGER	varchar(10)	TEXT	datetime	datetime	bigint
> 1		my article title	my article content			1

<그림1_DB_Many To One Relationships 02_기존 데이터에 추가된 user_id 필드 확인>

게시글 CREATE

게시글 CREATE 구현 (1/5)

- 새 게시글 작성 시 ArticleForm 출력 변화 확인
 - 새롭게 추가된 ForeignKey 필드인 User 필드 확인됨
- User 모델에 대한 사용자 입력 창이 나오지만 사용자가 입력하지 않아야 하는 입력
 - 다른 사람을 선택하게 되면 사칭에 대한 문제가 발생할 수 있음



게시글 CREATE 구현 (2/5)

- 기존 ArticleForm 에서 사용자가 입력할 수 있는 필드를 변경
 - 글 작성자는 사용자가 선택하지 않아도 되는 정보
- 사용자는 게시글 제목과 내용만 입력하도록 수정

```
# articles/forms.py
class ArticleForm(forms.ModelForm):
    class Meta:
        model = Article
        fields = ('title', 'content', )
```

게시글 CREATE 구현 (3/5)

- 게시글을 작성하면 아래와 같이 에러가 발생
 - NOT NULL constraint failed: articles_article.user_id
 - user_id 필드 데이터가 누락되어 NOT NULL 제약 조건에 위배



게시글 CREATE 구현 (4/5)

- 게시글 작성 시 작성자 정보가 함께 저장될 수 있도록 save의 commit 옵션 활용
 - 게시글 작성자는 현재 글을 작성하는 로그인 된 사용자(`request.user`) 정보를 저장

```
# articles/views.py

@login_required
def create(request):
    if request.method == 'POST':
        form = ArticleForm(request.POST)
        if form.is_valid():
            article = form.save(commit=False)
            article.user = request.user
            article.save()
            return redirect('articles:detail', article.pk)
    else:
        ... (중략)
```

게시글 CREATE 구현 (5/5)

- 게시글 작성 후 데이터베이스 articles_article 테이블 확인

	id	* title	* content	* created_at	* updated_at	* user_id
	INTEGER	varchar(10)	TEXT	datetime	datetime	bigint
> 1		my article title	my article content			1
> 2		title	content			1
> 3		web title	web content			2

<그림4_DB_Many To One Relationships 02_게시글 생성 확인>

※ 로그인한 User에 따라 저장되는 User PK가 다를 수 있음을 유의

게시글 READ

게시글 READ 구현 (1/2)

- index 페이지에 게시글 작성자 정보 출력하도록 수정

```
<!-- articles/index.html -->

{% for article in articles %}
  <p>작성자 : {{ article.user }}</p>
  <p>글 번호: {{ article.pk }}</p>
  <a href="{% url 'articles:detail' article.pk %}">
    <p>글 제목: {{ article.title }}</p>
  </a>
  <p>글 내용: {{ article.content }}</p>
  <hr>
{% endfor %}
```



〈그림5_DB_Many To One Relationships 02_작성자 정보 확인 1〉

게시글 READ 구현 (2/2)

- 각 게시글 상세 페이지에도 작성자 정보 출력하도록 수정

```
<!-- articles/detail.html -->
```

```
<h2>DETAIL</h2>
```

```
<h3>{{ article.pk }} 번째 글</h3>
```

```
<hr>
```

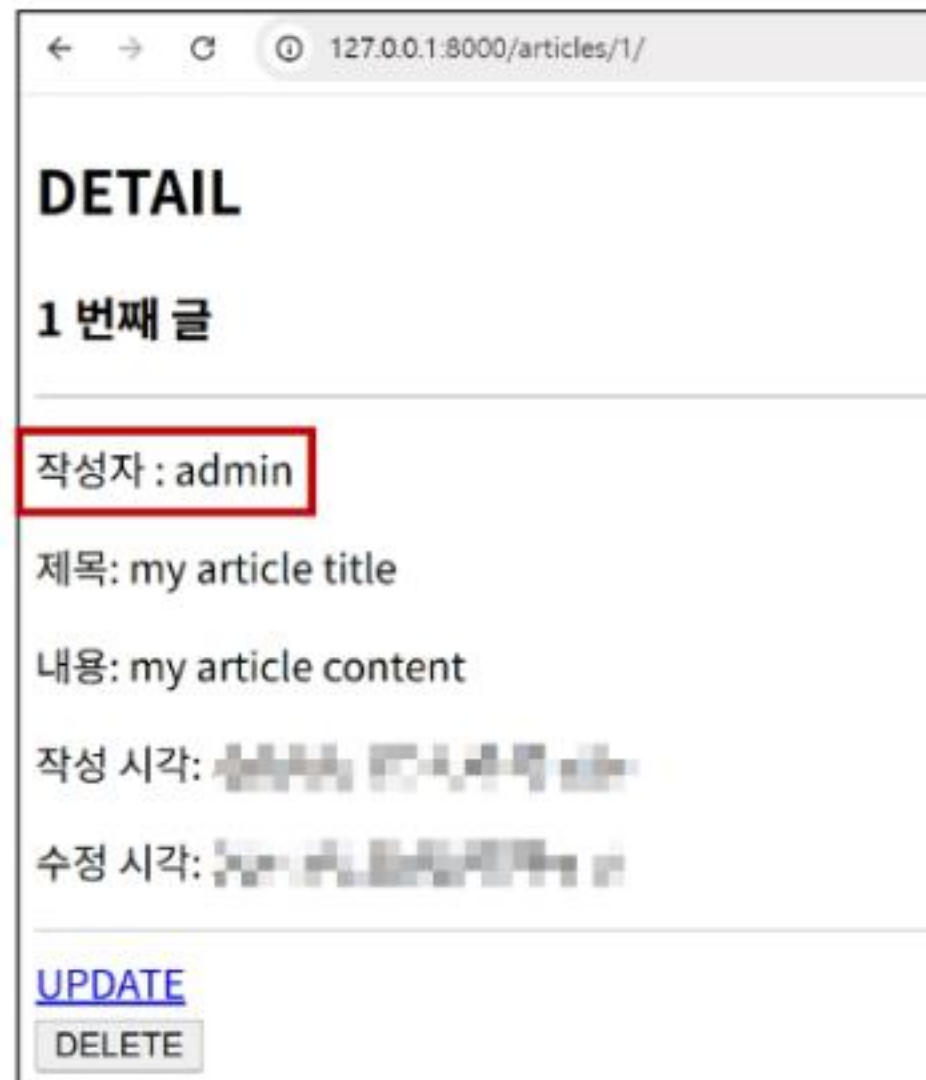
```
<p>작성자 : {{ article.user }}</p>
```

```
<p>제목: {{ article.title }}</p>
```

```
<p>내용: {{ article.content }}</p>
```

```
<p>작성 시각: {{ article.created_at }}</p>
```

```
<p>수정 시각: {{ article.updated_at }}</p>
```



〈그림6_DB_Many To One Relationships 02_작성자 정보 확인 2〉

게시글 UPDATE

게시글 UPDATE 구현 (1/2)

- 게시글 수정 요청 사용자와 게시글 작성 사용자를 비교하여 본인의 게시글만 수정 할 수 있도록 하기

```
@login_required
def update(request, pk):
    article = Article.objects.get(pk=pk)
    if request.user == article.user:
        if request.method == 'POST':
            form = ArticleForm(request.POST, instance=article)
            if form.is_valid():
                form.save()
                return redirect('articles:detail', article.pk)
        else:
            form = ArticleForm(instance=article)
    else:
        return redirect('articles:index')
    context = {
        'article': article,
        'form': form,
    }
    return render(request, 'articles/update.html', context)
```


게시글 UPDATE 구현 (2/2)

- 해당 게시글의 작성자가 아니라면, 수정/삭제 버튼을 출력하지 않도록 하기

```
<!-- articles/detail.html -->
```

```
{% if request.user == article.user %}
```

```
<a href="{% url 'articles:update' article.pk %}">UPDATE</a><br>
```

```
<form action="{% url 'articles:delete' article.pk %}" method="POST">
```

```
  {% csrf_token %}
```

```
  <input type="submit" value="DELETE">
```

```
</form>
```

```
{% endif %}
```


게시글 DELETE

게시글 DELETE 구현

- 삭제를 요청하려는 사람과 게시글을 작성한 사람을 비교하여 본인의 게시글만 삭제 할 수 있도록 하기

```
# articles/views.py

@login_required
def delete(request, pk):
    article = Article.objects.get(pk=pk)
    if request.user == article.user:
        article.delete()

    return redirect('articles:index')
```

TIP

views.py 에서 또 사용자를 구분하여 처리하는 이유

- 요청을 보내는 방법은 브라우저만 있는 것이 아니기 때문에 내부에서도 반드시 처리가 필요해요.
- requests 패키지 혹은 postman 어플을 이용한 요청은 브라우저를 통하지 않기 때문에 내부에서 사용자 구분을 해야 합니다.

Comment & User

모델 관계 설정

Comment - User 모델 관계 설정

- User 는 여러 개의 댓글을 작성
- Comment 는 한 명의 작성자가 작성
- Comment 모델 클래스에 User 필드를 참조하도록 외래 키(ForeignKey) 설정

```
# articles/models.py

class Comment(models.Model):
    article = models.ForeignKey(Article, on_delete=models.CASCADE)
    user = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.CASCADE)
    content = models.CharField(max_length=200)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)
```

Migration (1/2)

- 이전에 Article 와 User 모델 관계 설정 때와 동일한 상황
- 기존 Comment 테이블에 새로운 필드가 빈 값으로 추가 될 수 없기 때문에 기본 값 설정 과정이 필요

```
$ python manage.py makemigrations
```

```
# ... 공통 과정 생략
```

```
$ python manage.py migrate
```

Migration (2/2)

- Migration 후 articles_comment 테이블에 user_id 필드 확인

	id	content	created_at	updated_at	article_id	user_id
	INTEGER	varchar(200)	datetime	datetime	bigint	bigint
> 1		First comment			1	1
> 2		Second comment			1	1

〈그림7_DB_Many To One Relationships 02. 데이터베이스에 작성자 저장 확인〉

댓글 CREATE

댓글 CREATE 구현 (1/3)

- 댓글 작성 시 이전에 게시글 작성 할 때와 동일한 에러 발생
 - NOT NULL constraint failed: articles_comment.user_id
 - 댓글의 작성자를 저장하는 user_id 필드 데이터가 누락되었기 때문



댓글 CREATE 구현 (2/3)

- 댓글 작성 시 작성자 정보가 함께 저장될 수 있도록 작성

```
# articles/views.py

def comments_create(request, pk):
    article = Article.objects.get(pk=pk)
    comment_form = CommentForm(request.POST)
    if comment_form.is_valid():
        comment = comment_form.save(commit=False)
        comment.article = article
        comment.user = request.user
        comment.save()
        return redirect('articles:detail', article.pk)
    context = {
        ... (중략)
```

댓글 CREATE 구현 (3/3)

- 댓글 작성 후 articles_comment 테이블 확인

	id INTEGER	* content varchar(200)	* created_at datetime	* updated_at datetime	* article_id bigint	* user_ bigint
> 1		First comment			1	1
> 2		Second comment			1	1
> 3		Third comment with User			1	2

<그림9_DB_Many To One Relationships 02_댓글 데이터에 같이 저장된 user 확인>

※ 로그인한 User에 따라 저장되는 User PK가 다를 수 있음을 유의

댓글 READ

댓글 READ 구현

- 댓글 출력 시 댓글 작성자와 함께 출력

```
<!-- articles/detail.html -->
```

```
<h4>댓글 목록</h4>
```

```
<ul>
```

```
  {% for comment in comments %}
```

```
    <li>
```

```
      {{ comment.user }} - {{ comment.content }}
```

```
      ... (중략)
```

```
    </li>
```

```
  {% endfor %}
```

```
</ul>
```

댓글 목록

- admin - First comment

DELETE

- admin - Second comment

DELETE

- user01 - Third comment with User

DELETE

<그림10_DB_Many To One Relationships 02_댓글 작성자 확인>

댓글 DELETE

댓글 DELETE 구현 (1/2)

- 해당 댓글의 작성자가 아니라면, 댓글 삭제 버튼을 출력하지 않도록 함

```
<!-- articles/detail.html -->

{% for comment in comments %}
  <li>
    {{ comment.user }} - {{ comment.content }}
    {% if request.user == comment.user %}
      <form action="{% url 'articles:comments_delete'
        article.pk comment.pk %}" method="POST">
        {% csrf_token %}
        <input type="submit" value="DELETE">
      </form>
    {% endif %}
  </li>
{% endfor %}
```

댓글 목록

- admin - First comment
DELETE
- admin - Second comment
DELETE
- user01 - Third comment with User

Content:

제출

<그림11_DB_Many To One Relationships 02_댓글 삭제 버튼 확인>

댓글 DELETE 구현 (2/2)

- 댓글 삭제 요청 사용자와 댓글 작성 사용자를 비교하여 본인의 댓글만 삭제 할 수 있도록 하기

```
# articles/views.py

def comments_delete(request, article_pk, comment_pk):
    comment = Comment.objects.get(pk=comment_pk)
    if request.user == comment.user:
        comment.delete()

    return redirect('articles:detail', article_pk)
```


View decorators

View decorator

View decorator

View 함수의 동작을 수정하거나 추가 기능을 제공하는 데 사용되는 Python 데코레이터

View decorator는 일종의 "사전 체크리스트"처럼 동작해서 사용자 접근을 보다 안전하고 체계적으로 관리할 수 있습니다.

View decorator는 뷰 함수 위에 붙여서 작동하며, 코드 흐름을 방해하지 않고 깔끔하게 조건을 설정할 수 있습니다.

자주 쓰이는 View decorator는 로그인 여부, 권한 검사, HTTP 요청 방식 제한 등을 다룹니다.

View decorators 종류

- Allowed HTTP methods
 - 뷰가 허용하는 HTTP 요청 방식(GET, POST 등)을 제한
- Conditional view processing
 - 클라이언트가 보낸 조건을 확인한 후, 조건에 따른 응답 처리
- GZip compression
 - 서버에서 응답 데이터를 압축해서 전송
- ...
- 추가 view decorators 는 아래 공식 문서 참고
 - <https://docs.djangoproject.com/en/5.2/topics/http/decorators/>

Allowed HTTP methods

Allowed HTTP methods

Allowed HTTP methods

특정 HTTP method로만 View 함수에 접근할 수 있도록 제한하는 데코레이터

웹 요청은 GET, POST 등 다양한 요청 방식(method)을 사용하며, 각 방식은 고유한 목적을 갖고 있습니다.

요청 방식을 제한하면 보안이 강화되고, 코드의 역할도 명확하게 분리됩니다.

해당 데코레이터를 사용하면 View 로직을 보호하면서 사용자에게도 올바른 인터페이스를 제공할 수 있습니다.

주요 Allowed HTTP methods

- `require_http_methods(["METHOD1", "METHOD2", ...])`
 - 지정된 HTTP method만 허용
- `require_safe()`
 - GET과 HEAD method만 허용
- `require_POST()`
 - POST method만 허용

※ 허용되지 않은 method로 요청하는 경우 405 (Method Not Allowed) 오류 반환

Allowed HTTP methods 안내 (1/3)

- `require_http_methods(request_method_list)`
 - 지정된 HTTP method만 허용
 - 인자는 list 이며, 해당 list의 요소는 요청이 허용될 HTTP method 를 문자열(대문자)로 작성
- list에 작성된 method 이외의 method로 요청하면 `HttpResponseNotAllowed (405)` 를 반환

```
from django.views.decorators.http import require_http_methods

@require_http_methods(['GET', 'POST'])
def func(request):
    pass
```


Allowed HTTP methods 안내 (2/3)

- `require_safe()`
 - GET과 HEAD method만 허용
- GET과 HEAD method 이외의 method로 요청하면 `HttpResponseNotAllowed` (405) 를 반환

```
from django.views.decorators.http import require_safe

@require_safe
def func(request):
    pass
```

TIP

`require_GET` 이 있지만 `require_safe`를 권장하는 이유

- GET과 HEAD는 서버 상태를 변경하지 않고 조회를 수행하는 safe method로 정의돼 있어요.
- HEAD 요청은 주로 브라우저가 리소스를 받지 않고 메타 데이터 정보를 조회할 때 사용해요.
- 즉, `require_safe`가 `require_GET`보다 더 포괄적이며 표준 HTTP 클라이언트와의 호환성이 높아요.

Allowed HTTP methods 안내 (3/3)

- `require_POST()`
 - POST method만 허용
- POST method 이외의 method로 요청하면 `HttpResponseNotAllowed (405)` 를 반환

```
from django.views.decorators.http import require_POST

@require_POST
def func(request):
    pass
```

ERD

ERD

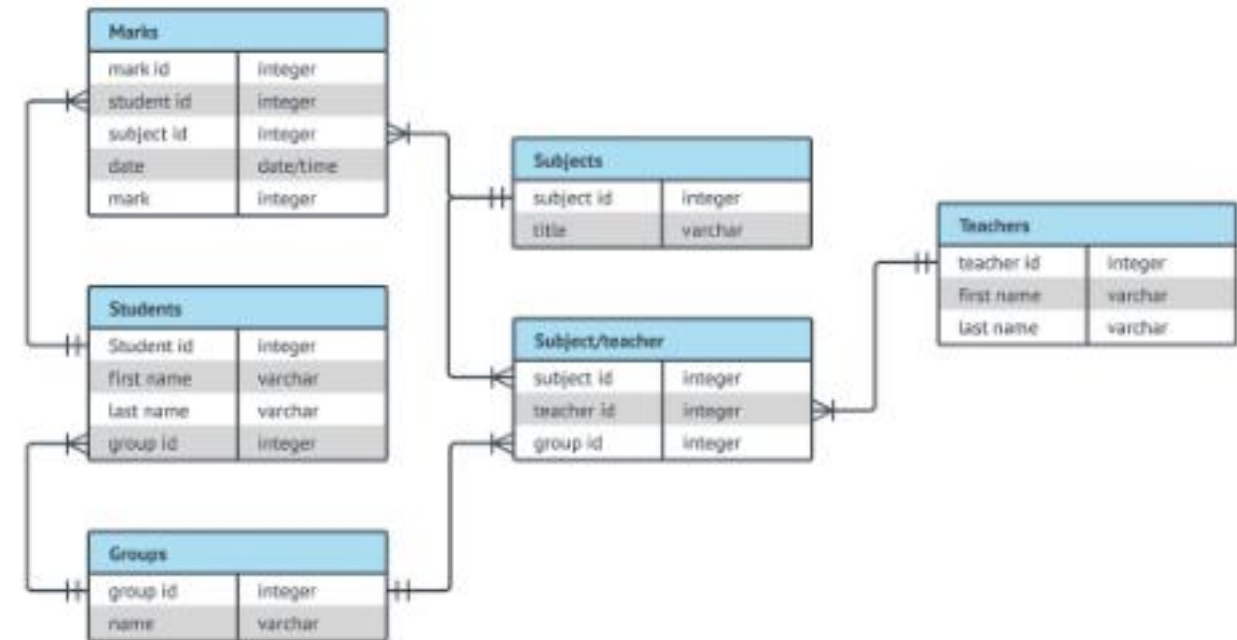
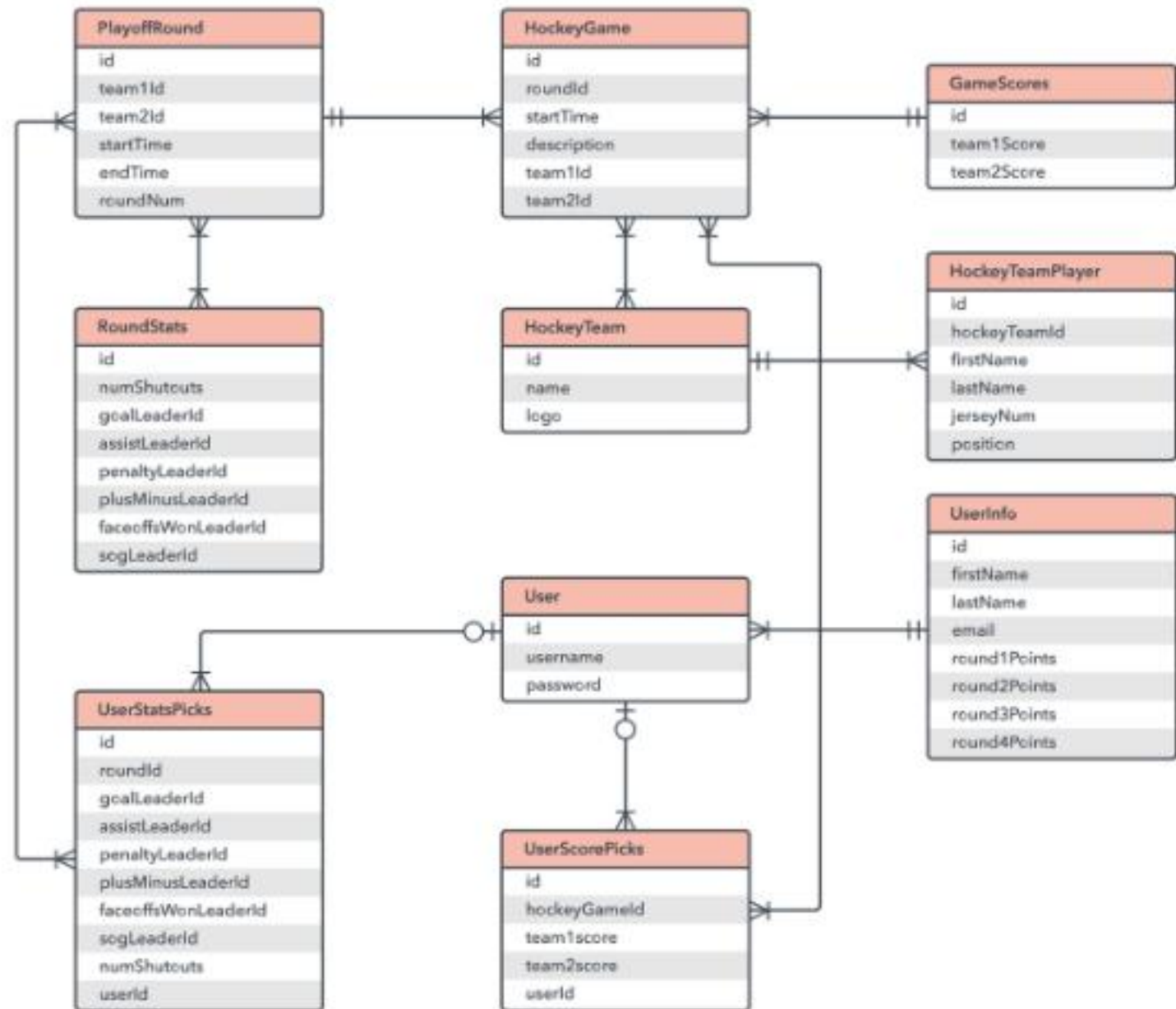
Entity-Relationship Diagram

데이터베이스의 구조를 시각적으로
표현하는 도구

Entity(개체), 속성, 그리고 엔티티 간의 관계를 그래픽 형태로 나타내어 시스템의 논리적 구조를 모델링하는 다이어그램입니다.

각 박스는 테이블을 나타내고, 그 안의 필드는 속성(컬럼), 화살표나 선은 관계(FK, 연관)를 나타냅니다. 이를 통해 개발자와 디자이너가 데이터 흐름을 쉽게 이해하고, 안정적인 DB 구조를 설계할 수 있습니다.

ERD 예시



ERD 구성 요소

ERD의 구성 요소

- 엔티티(Entity)
 - 데이터베이스에 저장되는 객체나 개념
 - 데이터베이스에서 주로 테이블로 표현됨
 - ex) '게시글', '댓글', '회원'
- 속성(Attribute)
 - 엔티티가 가지는 고유한 데이터 항목
 - 테이블의 컬럼으로 표현됨
 - ex) '게시글' 엔티티의 '제목', '내용', '작성일자', '생성일자'
- 관계(Relationship)
 - 엔티티 간의 연관성을 나타냄
 - 테이블 간의 연결된 선으로 표현됨
 - ex) 게시글을 '작성' 한 회원, 게시글에 '달린' 댓글

개체와 속성 표현 방법

- 개체 : 회원(User)
- 속성 : 회원번호(id), 이름(name), 주소(address) 등
 - 개체가 지닌 속성 및 속성의 데이터 타입

User	
id	integer
name	varchar
address	varchar

<표1_DB_Many To One Relationships 02_User 테이블>

관계

- 관계 : 회원과 댓글 간의 관계
 - 회원이 '작성'한 댓글

User	
id	integer
name	varchar
address	varchar

<표1_DB_Many To One Relationships 02_User 테이블>

Comment	
id	integer
content	varchar
created_at	datetime
user_id	integer

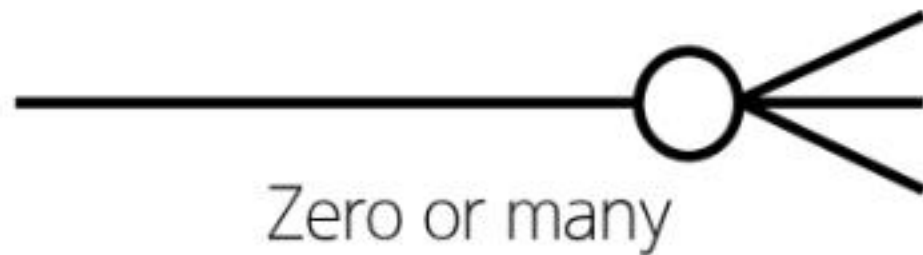
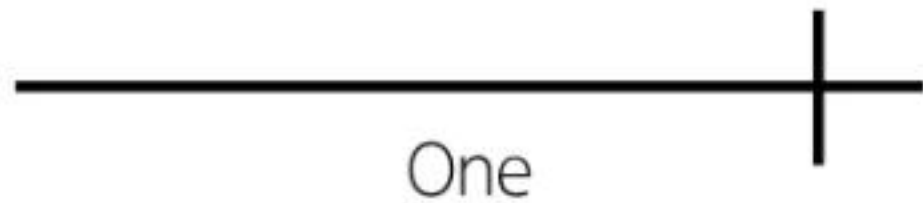
<표2_DB_Many To One Relationships 02_댓글 테이블>

Cardinality

- 한 엔티티와 다른 엔티티 간의 수적 관계를 나타내는 표현
- 일대일 (one-to-one, 1:1)
 - 한 엔티티의 레코드가 다른 엔티티의 하나의 레코드와만 연결되는 관계
 - ex) 한 '사람'은 하나의 '여권'만 소유하고, '여권'도 한 '사람'에게만 발급
- 다대일 (many-to-one, N:1)
 - 여러 레코드가 하나의 엔티티와 연결되며, 반대 방향에서는 하나만 연결되는 관계
 - ex) 여러 명의 '학생'이 하나의 '학교'에 소속됨
- 다대다 (many-to-many, M:N)
 - 양쪽 모두 여러 레코드와 연결되는 관계
 - ex) '학생'은 여러 '과목'을 수강할 수 있고, '과목'도 여러 '학생'이 수강할 수 있음

Cardinality 표현

- 선의 끝부분에 표시되며 일반적으로 숫자나 기호(Crow's foot)로 표현됨



<그림13_D8_Many To One Relationships 02_카디널리티 표현 (Crow's foot)>

Cardinality 적용

- 회원은 여러 댓글을 작성한다.
- 각 댓글은 하나의 회원만 존재한다.

User	
id	integer
name	varchar
address	varchar

<표1_DB_Many To One Relationships 02_User 테이블>



Comment	
id	integer
name	varchar
created_at	datetime
user_id	integer

<표2_DB_Many To One Relationships 02_댓글 테이블>

ERD의 중요성

- 데이터베이스 설계의 핵심 도구
 - 엔티티(테이블), 속성(컬럼), 관계를 명확히 정의함으로써 논리적 데이터 구조를 시각적으로 표현함
 - 복잡한 비즈니스 로직을 단순하고 직관적인 다이어그램으로 정리됨
 - 중복 데이터를 제거하고, 효율적인 저장 구조를 만들 수 있음
- 시각적 모델링으로 효과적인 의사소통 지원
 - 개발자, 기획자, 디자이너 등 다양한 직군이 ERD를 활용해 협업할 수 있음
 - 요구사항 분석 단계에서 누락된 기능이나 관계를 빠르게 파악 가능
 - 비전문가에게도 시스템 구조를 쉽게 설명할 수 있음
- 실제 시스템 개발 전 데이터 구조 최적화에 중요
 - 사전에 데이터 흐름과 연관 관계를 명확히 분석해 불필요한 관계나 비효율적인 설계를 방지함
 - 시스템 개발 전에 ERD를 작성함으로써, 이후 DB 구축 단계에서 중대한 구조적 오류를 줄이는 데 효과적임
 - 변경이나 유지보수 시에도 ERD를 기반으로 안정적으로 수정할 수 있음

ERD 제작 사이트

무료 ERD 제작 사이트

- Draw.io (diagrams.net)
 - 별도의 회원가입 없이 바로 사용 가능
 - 다양한 다이어그램 템플릿 제공
 - <https://app.diagrams.net/>
- ERDCloud
 - 실시간 협업 기능 지원
 - <https://www.erdcloud.com/>

참고

추가 기능 구현

인증된 사용자만 댓글 작성 및 삭제

- 로그인된 사용자만 댓글을 작성할 수 있도록 데코레이터 추가

```
# articles/views.py

@login_required
def comments_create(request, pk):
    pass

@login_required
def comments_delete(request, article_pk, comment_pk):
    pass
```

Article & User / Comment & User

- 3058. 할 일 목록 생성시 작성자 정보 추가하기
- 3059. 내 할 일 목록만 모아보기
- 1908. 리뷰 서비스 - 책과 리뷰와 유저
- 1903. 편의점 프랜차이즈 시스템 - 모델 정의
- 1905. 편의점 프랜차이즈 시스템 - 신규 매장과 신규 상품 등록
- 1906. 편의점 프랜차이즈 시스템 - 회원 정보 수정과 상품 삭제

인증된 사용자만 서비스 이용

- 3060. 1:N 과 로그인 기능을 활용한 코드 개선
- 1909. 리뷰 서비스 - 로그인 여부에 따른 리뷰 생성과 삭제
- 1904. 편의점 프랜차이즈 시스템 - auth 시스템

Q 각 문제에 올바른 답안을 생각해봅시다

1 Article이 User와 N:1 관계일 때 외래 키 설정은?

- a) ForeignKey('User', PROTECT)
- b) OneToOneField(User, CASCADE)
- c) ForeignKey(settings.AUTH_USER_MODEL, ...)
- d) ManyToManyField(User)

2 get_user_model() 함수는 주로 어디에서 사용하나요?

- a) models.py 내부
- b) 템플릿 파일
- c) models.py 외부 파일
- d) settings.py 내부

3 외래 키 필드 추가 시 필요한 설정은?

- a) null=True 설정
- b) 기본값 입력
- c) 모델 삭제
- d) related_name 지정

4 작성자 정보를 저장하려면 어떤 작업이 필요한가?

- a) form.save()
- b) commit=True 설정
- c) article.user = request.user
- d) article.save(commit=True)

Q 각 문제에 올바른 답안을 생각해봅시다

5 작성자 미지정 시 발생하는 에러는?

- a) KeyError
- b) NOT NULL constraint failed
- c) 404 Not Found
- d) CSRF token missing

6 @login_required를 사용하는 이유는?

- a) 사용자 정보를 삭제하기 위해
- b) 사용자 인증 없이 접근 허용
- c) 로그인한 사용자만 접근 허용
- d) 서버 오류를 방지하기 위해

7 require_POST의 기능은?

- a) GET만 허용
- b) POST만 허용
- c) 모든 요청 허용
- d) DELETE만 허용

8 ERD에서 엔티티는 무엇을 의미하나요?

- a) 테이블
- b) 외래 키
- c) HTTP 요청
- d) 사용자 인증

Q 각 문제에 올바른 답안을 생각해봅시다

9 ERD에서 속성은 무엇을 나타내나요?

- a) 테이블 이름
- b) 컬럼 정보
- c) 사용자 권한
- d) 관계 유형

11 댓글 작성자와 내용을 출력할 때 사용하는 코드는?

- a) `{{ user.comment }}`
- b) `{{ comment.content }}`
- c) `{{ comment.user }}` - `{{ comment.content }}`
- d) `{{ article.comment }}`

10 댓글 삭제 시 본인 확인을 위해 비교하는 값은?

- a) `comment.pk`와 `user.pk`
- b) `request.user`와 `comment.user`
- c) `article.pk`와 `comment.pk`
- d) `request.user`와 `article.user`

12 `request.user`와 비교하는 이유는?

- a) URL을 구분하려고
- b) 로그인 여부 확인
- c) 작성자 본인인지 확인
- d) 뷰 함수를 호출하려고





확인 문제



정답 및 해설

11. c) `{{ comment.user }}` - `{{ comment.content }}` / 12. c) 작성자 본인인지 확인 / 13. b) 테이블 구조 시각화
14. `request.user` / 15. c) 에러 없이 안전하게 참조하려고 / 16. `request.user`

11. 템플릿에서는 comment 객체의 user(작성자)와 content(내용) 속성을 함께 출력하여 댓글 정보를 표시합니다.
12. 작성자 본인만 수정 또는 삭제할 수 있도록 하기 위해 `request.user`와 객체의 user를 비교합니다.
13. ERD는 데이터베이스의 테이블과 관계를 시각적으로 표현하여 구조를 쉽게 이해하고 설계할 수 있게 돕는 도구입니다.
14. `request.user`는 현재 요청을 보낸 사용자이며, `comment.user`는 댓글을 작성한 사용자입니다. 두 값을 비교해 작성자 본인인지 확인합니다.

15. `settings.AUTH_USER_MODEL`은 모델 로딩 순서에 상관 없이 User를 안정적으로 참조할 수 있게 합니다.
16. 로그인한 사용자의 정보를 게시글의 작성자로 저장하기 위해 `article.user`에 `request.user`를 할당합니다.







